EDUHUB / ENGINEERING DOCUMENTATION

DEPLOYMENT STRATEGY

EduHub | Online Learning Platform

01 RELEASE
CI/CD, promotion and rollback

02 OPERATIONS
Monitoring, recovery and capacity

03 GOVERNANCE
Security, evidence and ownership

System Analysis & Design / Additional Deliverables

Candidate strategy based on the EduHub brief and established design baseline.

PROPOSED / NOV 2026

01 / DOCUMENT CONTROL

Deployment strategy & scope

Release engineering for a hosted, three-layer ASP.NET Core learning platform.

1	3	3
DEPLOYABLE .NET APP	LOGICAL APPLICATION LAYERS	REQUIRED USER ROLES

Purpose & decision status

This strategy explains **how EduHub is provisioned, built, released, operated, recovered and scaled** after its architecture is implemented. It is a candidate deployment design, not evidence that an Azure environment or CI/CD pipeline already exists.

Field	Candidate baseline
Document ID	EDH-SAD-DS-001
Version / status	1.0 | Proposed for team and supervisor review
System	EduHub - Online Learning Platform
Lifecycle stage	System Analysis & Design / Additional Deliverables
Architecture	N-Tier modular monolith: EduHub.Web, EduHub.Services, EduHub.Data
Delivery target	Azure App Service with managed Azure SQL / Blob dependencies
Source taxonomy	BRIEF: specified; ESTABLISHED: prior design; PROPOSED: decision requiring sign-off

Boundary Includes environment design, CI/CD, promotion, database release safety, operations and resilience. The companion UML Deployment Diagram remains the authoritative node-and-artifact view.

SOURCE BASIS [B1] Original 5-page brief | [E1] EduHub architecture and schema | [E2] Testing plan

02 / EXECUTIVE APPROACH

Deployment principles

A phased, auditable strategy appropriate for a graduation project and extensible to production.

Recommended release position

Begin with a **cost-aware capstone deployment** and make production safeguards explicit. Do not introduce microservices or Kubernetes to solve a problem the project does not have. Use one repeatable build artifact, isolated non-production data, and a human approval before any production release.

Decision	Approach	Status
Application boundary	One ASP.NET Core MVC deployable containing Web / Services / Data	ESTABLISHED
Database	Azure SQL Database; EF Core migration artifacts	ESTABLISHED / PROPOSED
Media	Private Azure Blob; authorized, short-lived playback access	BRIEF + PROPOSED
Payment	Stripe test mode in QA; production signed webhook verification	BRIEF + PROPOSED
Deploy	GitHub Actions with OIDC; promote tested artifact	PROPOSED
Release safety	Staging slot / swap when tier permits, with compatible schema	PROPOSED
Ops	Application Insights, alerts, backup + restore drill	PROPOSED

Guiding release principles

- Build once and promote the **same immutable package** across environments; inject environment-specific settings separately.
- No production database migration without a reviewed plan, tested backup/restore, and an explicitly authorized operator.
- Never grant course access based solely on Stripe checkout redirect; rely on verified payment-event fulfillment.
- Every deployment has a named owner, go/no-go decision, telemetry checks, and a rollback or forward-fix path.
- Expensive hardening features remain optional until load, security needs and budget justify them.

SOURCE BASIS [B1] | [M1] App Service slots | [M2] EF migrations | [M3] GitHub Actions

03 / ENVIRONMENTS

Environment and data isolation

Keep automated acceptance and integration testing away from production records.

Release environments are intentionally separate

Environment	Purpose & data	Entry policy
LOCAL	Developer machine; SQL Server local/dev DB; Stripe test keys; test Blob container.	Feature work; no shared secrets.
CI	Ephemeral build and test runners; synthetic fixtures and disposable test database.	Pull request; automated gates.
QA / UAT	Separate Azure App Service + QA SQL database + QA storage; Stripe test mode.	Successful CI build; tester approval.
PROD - staging slot	Pre-production slot of production app; production-compatible slot settings; nondestructive checks only.	Approved release; supported App Service tier.
PRODUCTION	Live user traffic, Azure SQL, restricted media, live Stripe; audited changes.	Manual approval + final validation.

Critical isolation rule QA is an independently deployed app and database. A production **staging slot is not a disposable QA environment**; during swap it can interact with production resources depending on slot settings. Prohibit seeded test purchases and destructive tests there.

Environment configuration contract

Configuration	LOCAL / QA	PRODUCTION
ASP.NET environment	Development / Staging	Production
Stripe account	Test mode; QA webhook signing secret	Live mode; dedicated production secret
Database	Local / isolated QA database	Production Azure SQL
Blob container	Synthetic QA videos	Private course media + certificates
Observability	Verbose sanitized diagnostics	Redacted structured logs + alerting

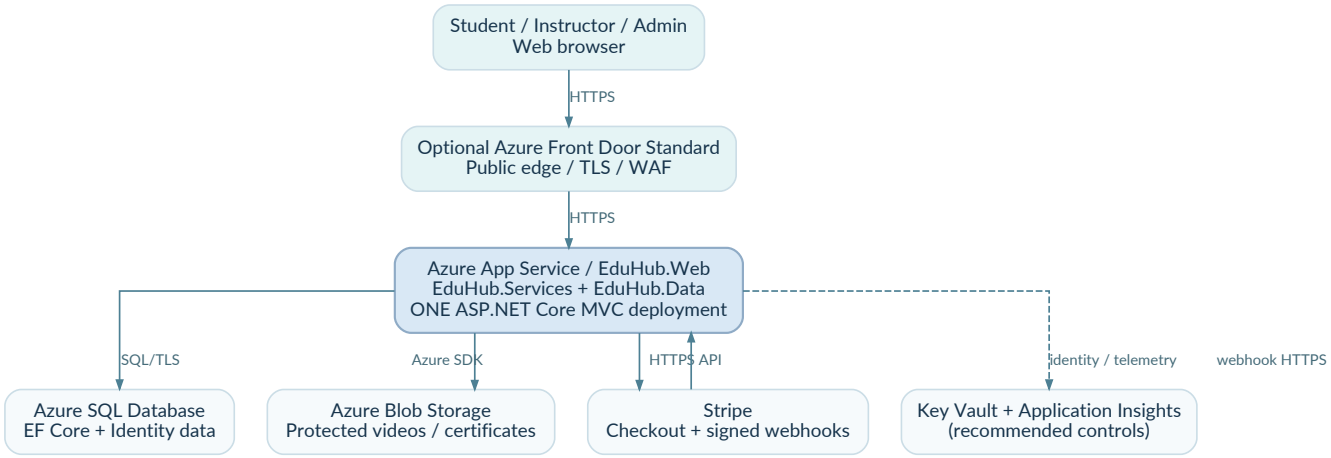
SOURCE BASIS [M1] App Service staging slots | [M4] Key Vault references

04 / TARGET TOPOLOGY

Core hosting architecture

Executable hosting decisions, not a repeat of the UML node-level document.

Proposed Azure deployment topology



Reading the diagram The three .NET projects compile into one hosted application. Azure SQL and Blob Storage are managed backing services; Stripe is an external provider. Front Door, Key Vault and telemetry are recommended or optional infrastructure, not additional application tiers.

SOURCE BASIS [B1] | [E1] Architecture/Deployment Diagram | [M5] Azure architecture

05 / AZURE INVENTORY

Azure resource responsibilities

Map each technology to an operational owner and lifecycle.

Service inventory & allocation

Resource	Stage / role	Baseline policy
App Service Plan	Hosts MVC app	Single plan sized for demonstration; Standard+ if using slots.
EduHub Web App	One deployment artifact	HTTPS only; Identity policies; health path.
Azure SQL Database	EF Core / ASP.NET Identity	Separate QA + production DB; backups enabled.
Storage Account	Videos, thumbnails, certificates	Private containers; narrow RBAC; secure SAS where required.
Stripe	Checkout and event confirmation	Test/live separation; validated signatures and durable event IDs.
Azure Monitor / Insights	Request/exception diagnostics	Correlation IDs; error alerts; synthetic uptime checks.
Azure Key Vault	Secrets custody	Recommended for live keys; managed identity app access.
Front Door Standard/Premium	Optional public/media edge	Scope and origin security reviewed before enabling.

Explicitly excluded from first release

- AKS/Kubernetes, service mesh, distributed transaction coordinator, and independent microservice deployments.
- Cross-region active-active traffic routing; consider only after disaster-recovery requirements are agreed.
- A dedicated video-transcoding worker is optional: the brief requires processing, but the processing engine and infrastructure are not specified.

Resource ownership Provision every resource in a tagged EduHub environment/resource group using repeatable IaC where feasible. Record region, SKU, responsible team member, retention, and estimated monthly cost.

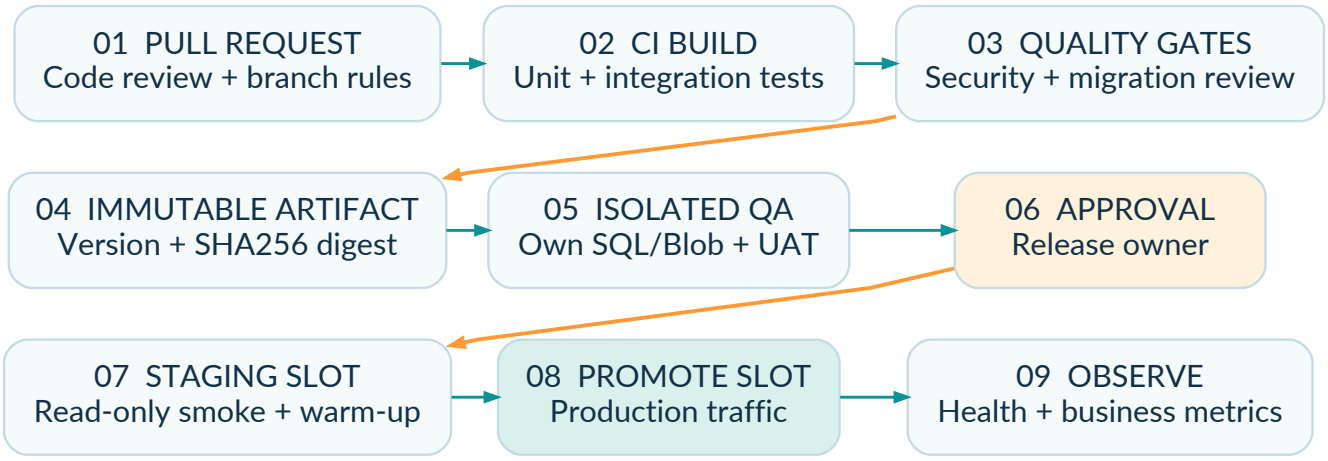
SOURCE BASIS [B1] | [E1] Deployment Diagram | [M5] Hosting plans

06 / DELIVERY PIPELINE

CI/CD release flow

Promote trusted artifacts through isolated validation and controlled production promotion.

From pull request to live release



Promotion policy The production-stage job must use an approved, previously tested artifact; rebuilding on production promotion risks deploying different bits. All production credentials and resources remain inaccessible to pull-request jobs.

Protected transitions

- Pull request to main: code review, green CI, security checks and a release tag.
- QA to production staging: successful UAT and quality-gate evidence, signed off by release owner.
- Staging to production: read-only smoke checks, dependency health, compatible schema and explicit approval.

SOURCE BASIS [M3] Azure App Service + GitHub Actions | [M1] Deployment best practices

07 / QUALITY GATES

Continuous integration gates

Deterministic tests, provider contract checks and secure artifact construction.

Build and test stages

Gate	Automated check	Failure response
G1. Compile	dotnet restore/build, warnings policy	Block PR / artifact.
G2. Unit	xUnit on pricing, progress, quiz, certificate	Block release.
G3. Integration	ASP.NET Core + SQL Server; auth, persistence, webhook fixture	Block release.
G4. Static security	Dependency audit, secrets scan, SAST	Block on critical findings.
G5. DB safety	Generate migration SQL/bundle; migration review	Hold for DBA/reviewer.
G6. Package	Publish MVC app, checksum and SBOM/manifest	Reject nonreproducible artifact.
G7. QA E2E	Student/instructor/admin journeys + Stripe test mode	No UAT sign-off.
G8. Prod smoke	Startup, catalog, auth, storage, read-only DB	No slot promotion.

Coverage and quality criteria

Use the previously drafted 86 functional/negative cases, 32 NFR checks and 12 UAT scripts as the candidate evidence library. These counts describe planned verification, not a completed test run. Prioritize payment integrity, role authorization, media access and database integrity over cosmetic defects.

> Separation of duties CI identity may build/test and publish artifacts. A production-bound deployment identity has only the scoped Azure permissions required, and environment protection rules require an authorized approver.

SOURCE BASIS [E2] Testing & Validation | [M3] GitHub Actions

08 / RELEASE GOVERNANCE

Release acceptance criteria

A documented decision rather than a deploy button.

Go / no-go checklist

Release criterion	Acceptance evidence	Authority
Critical tests green	All P0: login/roles, catalog, payment webhook, enrollment, streaming	QA lead
Required NFRs checked	NFR-01..07, 16, 18, 24, 26, 29, 30, 31, 32 as relevant	QA + architecture
Migration reviewed	Compatibility and data-impact review with rehearsal	Data owner
Rollback ready	Known-good artifact/slot available; no destructive schema rollback assumption	Release owner
Config verified	Environment settings and Stripe mode correct; secrets not logged	Operations
Resource health	DB and Blob accessible; App Insights telemetry active	Operations
Known defects assessed	No open critical/high severity defects without approval	Supervisor/team lead
UAT approved	Student, Instructor and Admin core workflows witnessed	Stakeholders

Proposed gate No production cutover if any critical security/payment/authorization check fails. Noncritical issues require explicit documented risk acceptance; this is a recommended policy, not an execution claim.

Release evidence bundle

Archive source commit SHA, artifact digest, deployment run ID, test reports, change log, migration script identifier, approval record, smoke-test evidence, and rollback decision. Never archive live secret values.

SOURCE BASIS [E2] Test plan | [M1] Deployment guidance

09 / ARTIFACT CONTROL

Versioning and promotion

Single artifact, predictable provenance, controlled rollback.

Branch, tag, artifact and version policy

Item	Recommended rule
Branch workflow	Short-lived feature branches; reviewed pull request into main.
Build trigger	PR: tests only. Main: protected artifact build and deploy to QA.
Release tag	vMAJOR.MINOR.PATCH or capstone milestone; include commit SHA.
Artifact	Publish output + manifest + checksum; store in restricted CI artifacts.
Migration	Version alongside application; track EF Core migration history.
Configuration	Environment settings remain outside immutable app package.
Promotion	Use exactly the tested artifact; do not rebuild to enter production.
Recovery	Retain last-known-good package and migration compatibility notes.

Build metadata inside EduHub

Expose a diagnostic version endpoint restricted to operators (or a health status dashboard) that returns application version, deployment SHA and non-sensitive runtime information. Keep public health endpoints minimal.

> Supply-chain rule Prefer GitHub OIDC federation over long-lived publish profiles. Restrict workflow permissions and pin production deployment to protected GitHub environments. [M3]

SOURCE BASIS [M3] GitHub Actions guidance | [E1] Technology Stack

10 / CONFIGURATION

Secrets and environment settings

No credentials in source, logs, downloadable packages or shared screenshots.

Configuration ownership and change control

Setting / credential	Custody	Deploy-time rule
ConnectionStrings:Default	Key Vault or protected App Service setting	Prod identity cannot modify schema.
Stripe:SecretKey	Environment secret	Never expose in browser or build logs.
Stripe:WebhookSecret	Distinct per webhook endpoint	Validate raw payload signature.
Blob storage access	Managed identity and narrow RBAC	Prefer user delegation SAS for limited client access.
Certificate generation	Licensing/configuration; private templates	No PII leakage in public URLs.
Feature flags	App settings or controlled store	Document default, owner and rollback.
Application Insights	Managed instrumentation config	Redact PII / payment details.

Secrets rotation runbook

01	Prepare Create new Stripe key or Key Vault secret without deleting the old one.
02	Activate Update relevant slot setting or secret reference and validate the target environment.
03	Verify Exercise a nonfinancial smoke check; for webhook secrets follow provider rotation overlap policy.
04	Retire Revoke old credential after observed stability; record who approved and when.

SOURCE BASIS [M4] Key Vault refs | [M6] Blob security | [M7] Stripe webhooks

11 / SECURITY BOUNDARIES

Network and media delivery

Prevent unauthorized access without forcing premature network complexity.

Network and protected-media rules

Boundary	Traffic	Required control
Browser -> MVC	HTTPS/443	TLS; anti-forgery; role / resource authorization.
MVC -> Azure SQL	TDS/TLS	Restricted firewall; optional private endpoint + VNet egress.
MVC -> Blob	Azure SDK over TLS	Managed identity; private containers; scoped RBAC.
Student -> media	HTTPS URL to media origin	Enrollment authorization and expiring, narrow SAS.
MVC <-> Stripe	HTTPS API + inbound webhook	Stripe test/live separation; signature, replay protection.
Edge -> origin	HTTPS Front Door if approved	Review origin access, token handling, and caching policy.

Important CDN caveat Protected paid videos must not become publicly cacheable because a signed URL appears in an edge cache. The simplest baseline is direct authorized Blob playback with a short-lived user delegation SAS; any Front Door media path requires a tested token-aware cache and origin-security design.

Optional network hardening

If budget and requirements justify it, add App Service outbound VNet integration and Azure SQL/Storage private endpoints with private DNS. This is not the same as placing the public web ingress behind a private endpoint; model inbound and outbound networking separately. [M8]

SOURCE BASIS [M6] Azure Storage security | [M8] Private PaaS networking

12 / DATABASE RELEASE

Schema migration strategy

Forward-compatible database evolution for a long-lived transaction system.

EF Core changes are independently deployed

Risk area	Safe deployment practice
Migration review	Generate idempotent SQL or EF bundle, inspect changes, rehearse against QA copy.
Expand/contract	Add compatible columns/indexes first; deploy new code; remove obsolete shape later.
Migration identity	Use least-privileged DDL-capable deployment principal; app runtime DML only.
Startup behavior	Do not rely on every App Service instance running schema mutations on startup.
Transactional data	Treat purchase, enrollment, payment and certificate records as durable business state.
Rollbacks	Reverting a code slot does not revert SQL schema; destructive Down() may lose data.

Release ordering

1	Review Approve forward-compatible SQL/EF bundle and expected runtime effects.
2	Protect Check Azure SQL restore point/backup capability, then record restore evidence.
3	Apply Run one authorized migration job; check migration history and constraints.
4	Promote Only after compatible schema validation, deploy application and monitor the critical flows.

Do not conflate methods The accompanying T-SQL schema is a design/reference artifact; EF Core migrations are the recommended actual change history once implemented. Avoid manually creating the entire schema and then blindly applying overlapping migrations.

SOURCE BASIS [M2] Applying EF Core migrations | [E1] Logical & Physical Schema

13 / MIGRATION RUNBOOK

Data-release rehearsal

Verify the schema on a realistic target before production receives traffic.

Database migration rehearsal checklist

Checkpoint	Expected evidence
Preflight	Migration files reviewed; target schema version known; drift check completed.
Restore readiness	Point-in-time recovery capability and correct Azure SQL region/server confirmed.
QA replay	Same migration applied to representative QA data, with duration recorded.
Integrity	Foreign keys, unique constraints, payment idempotency and row counts validated.
Permissions	Migration identity isolated from web runtime database permissions.
Mixed-version compatibility	Old and new application builds tolerate expanded schema during promotion.
Post-migration	Migration history consistent; no unexpected lock/performance symptoms.

Failure boundaries

- Pre-migration failure: stop and correct script; do not deploy new application.
- Migration failure before cutover: investigate partially applied work; repair forward or restore only with owner approval.
- Failure after users write new data: prefer a forward-compatible hotfix; database restoration may erase later purchases.
- Replayed Stripe events: maintain idempotency; reconcile payment and enrollment outcomes after any data recovery.

SOURCE BASIS [M2] EF Migrations | [M9] Azure SQL restore | [E2] Test plan

14 / RELEASE RUNBOOK

Production cutover procedure

A repeatable, sign-off-driven promotion of an immutable release.

Controlled deployment procedure

T-45	Freeze & announce Declare the release window, on-call owner, current artifact, open incidents and approved change set.
T-35	Validate dependencies Check SQL availability, Blob access, Stripe endpoint settings, Azure capacity and operational dashboards.
T-25	Backup & migration Confirm restore readiness; execute reviewed forward-compatible migration under limited deployment identity.
T-15	Deploy staging slot Publish the approved immutable package to the staging slot; verify slot-specific config.
T-08	Warm-up smoke Run /health, public catalog, role login and approved non-destructive media checks.
T-00	Approve & swap Recorded go/no-go; production slot swap; retain previously live app in staging slot.
T+10	Post-release smoke Verify live navigation, authorization, webhook 2xx path, metrics and no data anomalies.
T+30	Accept / revert Release owner records success, outstanding issues, observation window and rollback disposition.

Timings are illustrative T offsets are a runbook sequence, not execution-time promises. If the App Service tier has no slots, use an approved brief-maintenance deployment with a tested last-known-good artifact, or upgrade to a slot-capable tier.

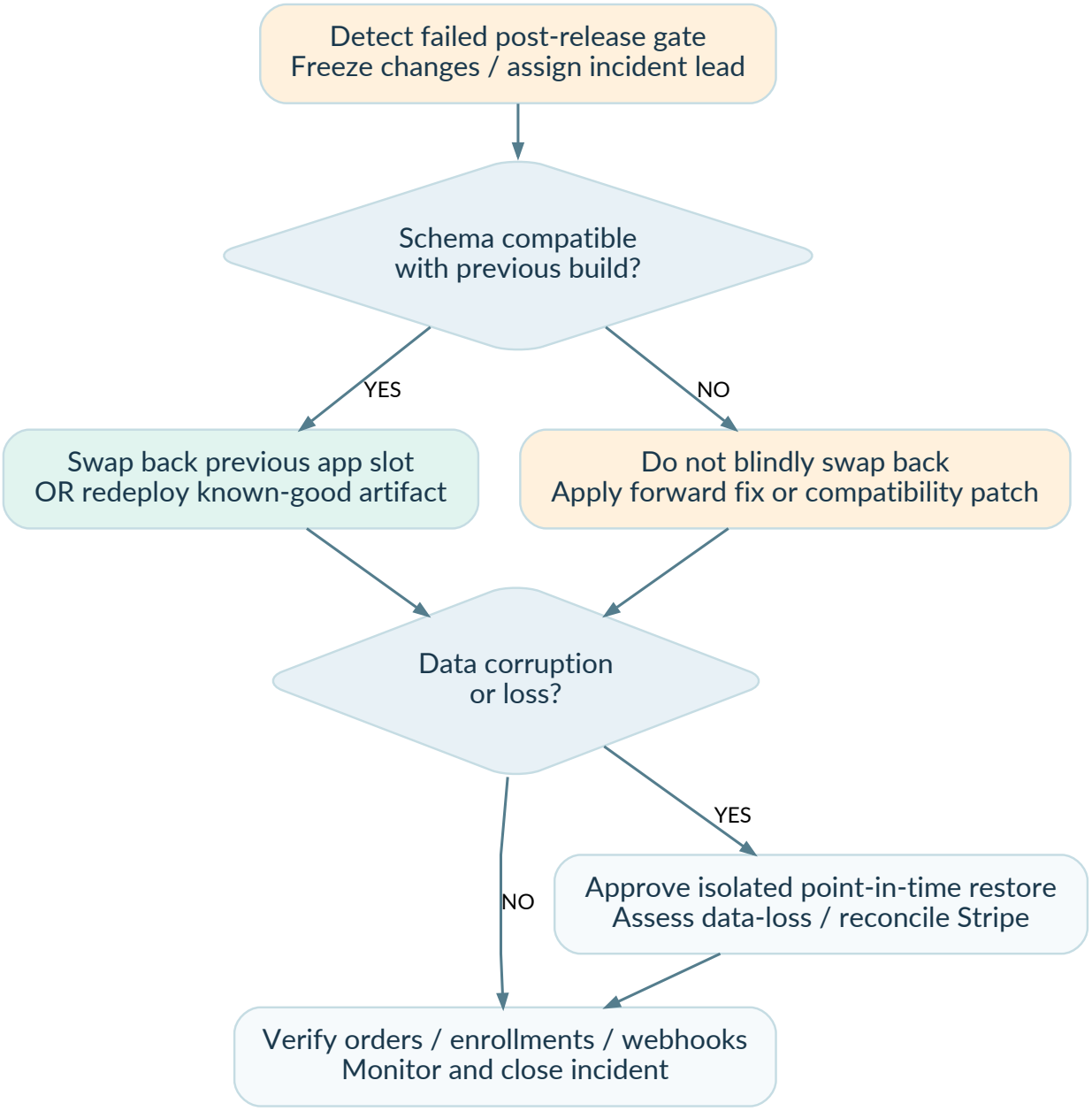
SOURCE BASIS [M1] Deployment slots | [M3] CI/CD

15 / ROLLBACK MODEL

Safe rollback decision tree

Protect payment and enrollment data when recovering from a faulty release.

Incident response: decide before reverting



Rollback distinction An App Service slot swap changes application content/configuration according to slot rules. It does not automatically restore the Azure SQL schema or reverse business data writes. A rollback is safe only after checking version/schema compatibility.

SOURCE BASIS [M1] Deployment slots | [M2] EF migration warnings | [M9] SQL restore

16 / INCIDENT RESPONSE

Rollback and forward-fix playbook

Recovery paths for runtime, data and provider failures.

Fault response matrix

Observed failure	First action	Recovery path
500 spike after swap	Disable promotion; trace last change	Swap back if schema compatible; else forward fix.
SQL migration defect	Stop deployment writes; assess schema history	Correct forward; controlled restore only with approval.
Stripe webhook errors	Keep provider retries; inspect signature/errors	Reprocess idempotently; reconcile orders.
Blob video access denied	Check RBAC/SAS expiration / CORS	Restore role/config; no broad public container.
Login/session issues	Inspect Identity keys/cookies and slot config	Fix config; avoid invalidating all sessions unnecessarily.
Slow queries / high load	Inspect Query Store, metrics, indexes	Scale plan or optimize query; throttle safely.
Compromised secret	Rotate immediately; isolate affected access	Invalidate keys; audit and notify owner.

Release-retention policy

Retain the previous validated application package and a change manifest for at least the current release window. Keep release logs, approvals and migration identifiers. Do not store customer PII or provider secrets in those artifacts.

> Escalation rule If restoring database data is required, explicitly assess RPO, downstream Stripe charges, missing enrollments, certificates, and already-issued links. Schedule reconciliation before declaring the incident closed.

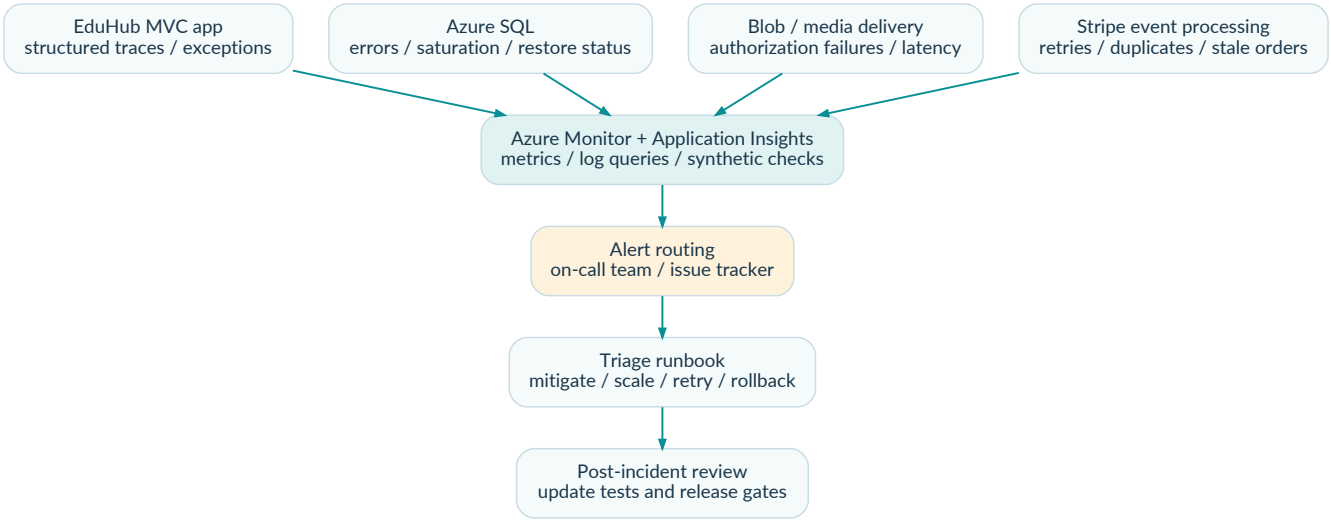
SOURCE BASIS [M1] | [M2] | [M9]

17 / OPERABILITY

Monitoring and alert topology

Observe the application, database, media delivery and external payment boundaries.

Operational observability pipeline



Logging boundary Record technical correlation identifiers and payment provider event IDs, not raw card data, authentication cookies, SAS URLs, student notes, or full certificate private fields.

SOURCE BASIS [M10] App Service operational guidance | [E2] Quality checks

18 / METRICS

SLIs, SLOs and logging

Make degraded checkout or playback visible before users report it.

Signals that explain real user impact

Area	Signal	Action trigger (proposed)
Web / requests	5xx count, p95 latency, health status	Sustained error growth; synthetic failure.
Identity	Denied auth spikes, lockouts, role escalation attempts	Unexpected privilege or attack pattern.
Stripe	Pending order age, signature failures, unprocessed event count	Backlog or repeated failures past threshold.
Data	SQL CPU, deadlocks, timeouts, failed transactions	Persistent saturation or integrity exception.
Media	Playback failures, 403/404, SAS errors, startup latency	Paid-user playback degraded.
Certificates	Generation failures, duplicate issuance, verification failures	Eligibility or uniqueness regression.
Capacity	Memory/CPU, request queue, thread pool, scale events	Sustained capacity pressure.

Dashboard layout

- Service health: availability, p95 requests, top errors and release markers.
- Business integrity: successful checkout event-to-enrollment latency, failed reconciliation queue, certificate issuance.
- Dependencies: Azure SQL, Blob response times, Stripe API and webhook delivery.
- Release panel: current version, error trend since promotion and active incident owner.

Telemetry design Instrument requests and domain workflows with a shared trace/correlation ID. Use Azure Monitor / Application Insights as the recommended observability stack and restrict access to sensitive logs.

SOURCE BASIS [M10] Well-Architected App Service | [E2] Test Plan

19 / QUALITY TARGETS

Performance and reliability gates

Proposed values preserved from the prior requirements baseline.

Candidate SLOs from EduHub requirements

Reference	Proposed target	Verification / monitoring
NFR-09	Catalog p95 <= 2.5 s at 50 concurrent test users	Repeatable 10-minute load test.
NFR-10	Dashboard p95 <= 3.0 s at 30 concurrent users	Controlled load + telemetry.
NFR-11	Video first frame p95 <= 5 s on 10 Mbps link	Browser network/performance test.
NFR-13	50 concurrent sessions with <= 1% failed operations	Representative mixed workload.
NFR-14	Availability >= 99.5% over 30 days (could-have)	Synthetic HTTPS probe; scope exclusions defined.
NFR-15	Capstone RPO <= 24 h; RTO <= 8 h	Timed representative restore drill.

Targets are proposals These numbers come from the previously drafted requirements and have not been demonstrated by testing. Do not claim a contractual Azure SLA or measured system performance without observed evidence.

Alert design method

Set each alert threshold only after a QA baseline. Favor impact signals (failed enrollment after paid event, sustained 5xx, inaccessible videos) over isolated CPU spikes. Associate each alert with an owner, severity and a direct runbook link.

SOURCE BASIS [E1] SRS NFR-09..15 | [E2] NFR validation | [M10]

20 / BUSINESS CONTINUITY

Backups and recovery objectives

Recover code, database, media and payment consistency as one business service.

Recovery policy and disaster boundaries

Asset / service	Recovery mechanism	Caveat
Azure SQL	Automated backups; point-in-time restore to a new DB	Reconcile changes after recovery point.
App Service code	Known-good build artifact / staging slot	Requires compatible DB shape.
Blob media	Versioning / soft delete / storage redundancy as approved	Check storage tier and retention before claims.
Stripe transactions	Provider records plus local event IDs	Must reconcile authoritative payment status.
Secrets	Key Vault recovery/rotation procedures	Restored apps must not use revoked keys.
Monitoring data	Configured telemetry retention	Not a substitute for business data backup.

RPO / RTO choices

The earlier SRS proposes RPO 24 hours and RTO 8 hours for the capstone environment; these are design targets, not achieved results. Azure SQL point-in-time restore creates a restored database; recovery also requires reconnection, application verification and Stripe reconciliation. [M9]

> Financial data principle Do not restore production data casually to pass a demo. A restore can remove locally recorded orders even when external Stripe charges remain successful. Protect an external reconciliation ledger or replayable event trail.

SOURCE BASIS [M9] Azure SQL restore | [E1] NFR-15

21 / DISASTER RECOVERY

Backup/restore rehearsal

A real drill is required before recovery targets can be described as achieved.

Quarterly or pre-demo recovery drill (proposed)

01	Define failure Assume accidental deletion or schema corruption in isolated QA; record incident time.
02	Select restore point Choose a point-in-time within configured retention; document expected lost changes.
03	Restore separately Recover a new Azure SQL DB, never overwrite live DB in place during the drill.
04	Reconfigure QA Point a QA app to the restored DB using protected settings; verify secret access.
05	Reconcile Check sample orders/enrollments/certificates and simulated Stripe event replay.
06	Validate Run sign-in, browsing, playback authorization and read-only certificate verification.
07	Measure Record RPO/RTO achieved, blockers, costs and revisions to the runbook.

Required restore evidence

Evidence item	Minimum record
Timing	Incident declared, restore initiated, service verified, total duration.
Integrity	Row counts, key constraints, schema version, payment reconciliation.
Availability	Synthetic health and functional smoke results.
Approval	Data owner acceptance and exceptions documented.

SOURCE BASIS [M9] SQL restore | [E2] NFR-15 testing

22 / CAPACITY

Scaling and performance strategy

Scale based on measured bottlenecks, not assumptions or theoretical peak traffic.

Capacity controls by growth stage

Stage	App / DB strategy	Video / media strategy
Capstone demo	Single modest App Service plan; small isolated database; test workloads.	Private Blob playback; optimize files and use browser player.
Early production	Enable health checks and scale-up; add 2nd instance where tier and budget allow.	Measure throughput and unauthorized access; evaluate edge delivery.
Demand growth	Autoscale App Service plan based on measured traffic; profile SQL queries/indexes.	Use approved CDN/Front Door route with verified protected-origin design.
Advanced	Cache bounded read models; background media processing if needed.	Segregate transcoding workload only when observed demand justifies.

Capacity verification before scaling

- Run the pre-agreed NFR-09/10/11/13 load profiles with documented data volume and region.
- Measure App Service CPU/memory together with Azure SQL utilization and slow query traces.
- Keep video byte transfer off the MVC server; do not scale the web process to deliver media.
- Scale out the App Service plan with awareness that plan-level instances may host multiple apps or slots.

Tier caution Free/Shared tiers do not provide the same scaling and staging capabilities as higher tiers. Choose a plan and region only after checking current Azure quotas and pricing; no unverified cost estimate is asserted here.

SOURCE BASIS [M5] Hosting plans | [M11] Autoscale | [E1] NFR-32

23 / DELIVERY OPTIONS

Cost-aware hosting profiles

Options rather than pretending one cloud tier fits every student project.

Two deployment profiles for team approval

Capability	CAPSTONE / cost-aware	PRODUCTION-READY / optional
Host	One basic App Service / single instance if available	Standard/Premium with staging slot + scale options
SQL	Separate small QA/prod DBs, minimum needed capacity	Capacity-tested Azure SQL tier; private endpoint if justified
Blob	Private containers + scoped SAS	Redundancy/versioning/edge media delivery as approved
Secrets	Protected app settings; no repo secrets	Key Vault + managed identity + rotation policy
CDN	Design and test secured media path; can defer activation	Approved Front Door Standard/Premium integration
Monitoring	Health endpoint + basic Application Insights	Dashboards, alerts, synthetic monitoring, retention
Deploy	Controlled deployment with downtime or manual stage	Slot warm-up, approved swap and reverse swap

Cost governance Retain a small Azure budget register with App Service plan, SQL compute/storage, Blob storage/egress, Front Door, telemetry ingestion and backup retention. Assign budget alerts and clean up unused QA resources after the review.

Azure CDN lifecycle decision

The original brief calls for Azure CDN; the established Technology Stack reviewed Azure Front Door Standard/Premium as a modern alternative. Approve the exact edge SKU and media security model before committing to it. Avoid deploying a retiring classic Front Door tier. [M12]

SOURCE BASIS [B1] Azure CDN requirement | [E1] Technology Stack | [M12] Front Door lifecycle

24 / GOVERNANCE

Team roles and RACI

A clear operational owner is more useful than a complex organization chart.

Deployment responsibility assignment

Work item	App dev	Data lead	QA lead	Release owner
Build/test pipeline	R	C	C	A
Schema migration review	C	R/A	C	I
QA + UAT evidence	C	C	R/A	I
Secrets and Azure IAM	C	C	I	R/A
Production promotion	C	C	C	R/A
Rollback / incident triage	R	R	R	A
Restore drill	C	R	C	A
Post-release signoff	C	C	R	A

RACI legend

R Responsible for execution. A Accountable for the decision. C Consulted. I Informed. One person may occupy multiple roles in a graduation team, but record a single accountable owner for production changes.

Change control For every production release record: ticket, version, change summary, database impact, planned window, named approver, test evidence, and rollback strategy.

SOURCE BASIS [E1] Architecture and Test Plan | [M1] App Service deployment

25 / SECURITY CHECKLIST

Secure deployment controls

Security requirements follow the artifact through promotion and operation.

Deployment security baseline

Control	Implementation and test evidence
Environment isolation	Separate Stripe modes, SQL DB, storage paths and app settings; negative test.
Azure RBAC	Minimal scopes for CI identities, developers and runtime managed identities.
TLS and domain	HTTPS-only App Service; validate certificate renewal and redirects.
Request safeguards	ASP.NET Identity, antiforgery, input validation, resource-owner authorization.
Webhook trust	Verify signed raw Stripe payload; unique event IDs and idempotent effects.
Media authorization	No public videos; short-lived SAS and ownership/enrollment checks.
Logging hygiene	No credentials/PII in telemetry; redact SAS and session tokens.
Supply chain	Dependency audit, source review, immutable artifacts, protected CI environments.

Post-deploy audit Validate production configuration using a checklist and minimum-privilege test accounts; do not assume a successful health probe proves correct payment authorization or data isolation.

SOURCE BASIS [E1] NFR-01..08 | [M3] CI/CD | [M6] Storage security

26 / AUTOMATION TEMPLATE

CI/CD implementation guidance

A practical bridge from architecture diagrams to repository automation.

Engineering commands: illustrative only

The companion source bundle includes an editable workflow template. The final paths, service principal / OIDC configuration and resource names must match the team repository and Azure subscription.

Stage	Representative command / action
Restore + test	dotnet restore; dotnet test --configuration Release
Publish MVC	dotnet publish EduHub.Web/EduHub.Web.csproj -c Release -o publish
Generate migration SQL	dotnet ef migrations script --idempotent
Build EF bundle	dotnet ef migrations bundle
Deploy to slot	azure/webapps-deploy@v3 / slot-name staging
Promote after gates	az webapp deployment slot swap with reviewed arguments

Automation safeguards

- Store Azure identifiers in CI environment variables and credentials as protected OIDC configuration.
- Use QA environment as the acceptance target; production promotion job requires approval and invokes a separate stage.
- Run migration only once from a gated job; never from every web instance start-up.
- All template commands must be dry-run and reviewed with actual app names, SQL connection policy, and Azure roles.

No false execution claim The included workflow is an editable template. It is not connected to a repository or Azure subscription and has not executed against live EduHub.

SOURCE BASIS [M3] GitHub Actions | [M2] EF migrations

27 / VALIDATION

Deployment acceptance evidence

A measurable definition of deployment readiness and project defense evidence.

Deployment acceptance traceability

Baseline	Deployment verification	Evidence artifact
NFR-24 / maintainability	Enforce N-Tier project dependency rules in build	Architecture test report
NFR-26 / integrations	Auth, payments, SQL, media contract tests run	Integration CI log
NFR-27 / observability	Trace ID across controller/services/persistence	Sanitized telemetry trace
NFR-28 / configuration	Same binary operates with QA vs prod settings	Configuration diff / manifest
NFR-29 / Azure deployment	Clean staging publish and healthy route	App Service deployment log
NFR-30 / migrations	Clean and forward EF migration succeeds	Migration run + DB history
NFR-31 / provider failures	Controlled Stripe/Blob outage responses	Fault-injection result
NFR-32 / media scaling	Video bytes served via authorized Blob / edge path	Browser network evidence
NFR-15 / restore	Target restore time and recovery point drill	DR report

Evidence states

Designed = test/runbook exists; Executed = actual dated run; Passed = recorded outcome meets accepted threshold; Approved = designated owner signs release. This document is at the Designed state only.

Review rule Reference IDs and targets match the previous EduHub SRS/Test Plan; any scope change requires updating requirements, test cases and this deployment strategy together.

SOURCE BASIS [E1] SRS | [E2] Testing & Validation

28 / IMPLEMENTATION

Phased deployment roadmap

A sequence the project team can convert into sprint tasks and a release board.

Implementation roadmap: proposed sequence

Milestone	Activities	Exit evidence
M1. Provision	Azure subscription/RG, app plan, QA SQL/Blob, test Stripe endpoint	Recorded resource inventory
M2. Secure	Identity, secrets store, HTTPS, RBAC, signed media rules	Security checklist + negative tests
M3. Build CI	Compile, unit/integration, scan, immutable publish	Green pipeline and digest
M4. QA/UAT	Deploy to isolated QA, validate 3 roles, test payments and video	UAT and quality report
M5. Prod rehearsal	Migration, recovery drill, staging smoke, test promotion	Signed release runbook
M6. Launch + observe	Approved cutover, post-release smoke, dashboards, incident drills	Acceptance report + action list

Release gates by phase

M1-2	M3-4	M5-6
FOUNDATION	VERIFICATION	PROMOTION

Scheduling Milestones describe dependency order, not promised calendar dates or a claimed completion percentage. Team capacity, Azure credits and course deadlines determine an achievable schedule.

SOURCE BASIS [B1] Original four-week project plan | [E2] Testing plan

29 / RISKS & APPROVALS

Risks, tradeoffs and open decisions

A decision-ready deployment plan must expose uncertainty explicitly.

Top deployment risks and responses

Risk	Impact / mitigation	Owner
R-01: DB schema breaks older app	High / expand-contract; migration rehearsal; no blind rollback	Data lead
R-02: Stripe duplicate/out-of-order events	High / signed webhook; durable event IDs; reconciliation	App lead
R-03: Confidential media leaked	High / private containers; expiring SAS; edge cache review	App lead
R-04: Secrets in build logs	High / OIDC + Key Vault; redaction/scan	Release owner
R-05: Expensive resources	Medium / budget alerts, SKU review and cleanup	Release owner
R-06: Slot config drifts	Medium / sticky settings audit, smoke checks	Operations
R-07: No usable recovery plan	High / scheduled representative restore drill	Data lead

Decision register requiring sign-off

- Azure region, App Service SKU/slot availability, SQL tier and resource budget.
- Exact CDN / Front Door secure-media design and whether it is enabled for demonstration.
- Database backup retention and actual restore/RPO/RTO targets.
- Production release approver, monitoring thresholds, rotation intervals and rollback window.
- CI platform and OIDC setup in the actual student team repository.

Document status These are deliberate open decisions, not contradictions in the source brief. Their final values should be recorded as approved ADRs or release configuration notes.

SOURCE BASIS [B1] | [M1-M12]

30 / REFERENCES A

References & provenance - Part I

Project sources, application lifecycle, EF migrations, identity and storage.

References and evidence hierarchy

The five-page source brief defines the scope; earlier EduHub design deliverables preserve decisions; official vendor documentation informs infrastructure implementation. Source details were reviewed as of 05 November 2026.

Ref	Source / link
[B1]	EduHub - Online Learning Platform, supplied project brief (5 pages) User-supplied project source; original requirements and mandated technologies.
[E1]	EduHub Software Architecture; Technology Stack; Deployment Diagram; Logical/Physical Schema; Requirements Prior EduHub candidate design deliverables, prepared in this conversation.
[E2]	EduHub Testing & Validation Prior candidate test plan and traceability; no tests have been run.
[M1]	Azure App Service deployment best practices https://learn.microsoft.com/en-us/azure/app-service/deploy-best-practices
[M2]	EF Core - Applying Migrations https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/applying
[M3]	Deploy to Azure App Service using GitHub Actions https://learn.microsoft.com/en-us/azure/app-service/deploy-github-actions
[M4]	Key Vault references for App Service app settings https://learn.microsoft.com/en-us/azure/app-service/app-service-key-vault-references
[M5]	Azure App Service plans and scaling https://learn.microsoft.com/en-us/azure/app-service/overview-hosting-plans
[M6]	Azure Blob Storage security recommendations https://learn.microsoft.com/en-us/azure/storage/common/storage-security-guide

Priority rule When the external implementation documentation differs from the original capstone brief, keep the source requirement visible and record any alternative as an explicit team decision.

31 / REFERENCES B

References & provenance - Part II

Official cloud operations, payment, recovery and scaling references.

External integration and operations references

Ref	Source / link
[M7]	Stripe Webhooks overview / event delivery https://docs.stripe.com/webhooks
[M8]	Azure private access to PaaS services https://learn.microsoft.com/en-us/azure/networking/design-guide/private-platform-as-a-service
[M9]	Azure SQL - Restore database from backup https://learn.microsoft.com/en-us/azure/azure-sql/database/recovery-using-backups
[M10]	Azure Well-Architected guidance for App Service https://learn.microsoft.com/en-us/azure/well-architected/service-guides/app-service-web-apps
[M11]	Azure Monitor autoscale https://learn.microsoft.com/en-us/azure/azure-monitor/autoscale/autoscale-get-started
[M12]	Azure Front Door classic retirement and migration https://learn.microsoft.com/en-us/azure/frontdoor/classic-retirement-faq

Evidence interpretation

BRIEF = project-stated; ESTABLISHED = previous EduHub design direction; PROPOSED = engineering recommendation. No Azure resource, GitHub workflow, Stripe integration, or quality gate has been configured or executed as part of this documentation.

Review checkpoint The team must finalize Azure region, tier, data retention, CDN strategy, production approvers and restore objectives before implementing these procedures.